

[Day 1] 종합 실습 결과 정리

판교 5반 홍은비

1. 시스템 개요 - 학사관리시스템_강의 수강 및 과제 관리 시스템

1.1 목적

학생의 강좌 수강정보와 강좌별 과제 및 제출 결과를 관리한다.

1.2 사용자 정의

사용자	역할
학생	수강 강좌와 과제를 확인하고 과제를 제출한다.
교수자	강좌에 과제를 등록하고 제출 결과를 확인·채점한다.

1.3 기능 요구사항

ID	기능	요구사항	사용자
REQ-01	학생 관리	학생의 학번, 이름, 이메일, 전화번호, 재학 여부를 관리한다.	교수자
REQ-02	강좌 관리	강좌 코드와 강좌명을 관리한다.	교수자
REQ-03	수강신청 관리	학생과 강좌의 수강 관계를 등록하고 조회한다.	학생·교수자
REQ-04	과제 관리	강좌별 과제 제목과 마감 일시를 등록한다.	교수자
REQ-05	과제 제출	학생의 과제 제출 일시를 기록한다.	학생
REQ-06	채점	제출물의 점수와 피드백을 기록한다.	교수자
REQ-07	제출 결과 조회	제출 시간과 마감 시간을 비교하여 정상·지각 제출 여부를 조회한다.	학생·교수자

2. 데이터베이스 설계

2.1 테이블 목록

테이블	용도
students	학생 정보
courses	강좌 정보
enrollments	학생과 강좌의 수강 관계
assignments	강좌별 과제
submissions	학생의 과제 제출 및 채점 결과

2.2 테이블 정의서

• students

컬럼	자료형	키-제약조건	기본값	설명
id	BIGINT	PK, IDENTITY	자동 증가	학생 식별자
student_no	VARCHAR(20)	NOT NULL, UNIQUE		학번
name	VARCHAR(50)	NOT NULL		학생 이름
email	VARCHAR(100)	NOT NULL, UNIQUE		이메일
phone	VARCHAR(20)	NULL 허용	NULL	전화번호
enrolled	BOOLEAN	NOT NULL	TRUE	재학 여부
created_at	TIMESTAMPZ	NOT NULL	CURRENT_TIMESTAMP	학생 등록 일시

• courses

컬럼	자료형	키-제약조건	기본값	설명
id	BIGINT	PK, IDENTITY	자동 증가	강좌 식별자
course_code	VARCHAR(20)	NOT NULL, UNIQUE		강좌 코드
title	VARCHAR(100)	NOT NULL		강좌명

• enrollments

컬럼	자료형	키-제약조건	기본값	설명
student_id	BIGINT	PK, FK, NOT NULL		학생 식별자
course_id	BIGINT	PK, FK, NOT NULL		강좌 식별자
enrolled_at	DATE	NOT NULL	CURRENT_DATE	수강신청일

• assignments

컬럼	자료형	키-제약조건	기본값	설명
id	BIGINT	PK, IDENTITY	자동 증가	과제 식별자
course_id	BIGINT	FK, NOT NULL		강좌 식별자
title	VARCHAR(200)	NOT NULL		과제 제목
due_at	TIMESTAMPZ	NOT NULL		제출 마감 일시

• submissions

컬럼	자료형	키-제약조건	기본값	설명
id	BIGINT	PK, IDENTITY	자동 증가	제출물 식별자
assignment_id	BIGINT	FK, NOT NULL		과제 식별자
student_id	BIGINT	FK, NOT NULL		학생 식별자
submitted_at	TIMESTAMPZ	NOT NULL	CURRENT_TIMESTAMP	제출 일시
score	NUMERIC(5,2)	NULL 허용, CHECK	NULL	점수
feedback	TEXT	NULL 허용	NULL	피드백

2. 실행 결과 화면 Capture

2.0 postgresSQL 접속 결과 화면

```
➤ + sql git:(main) ✕ psql -d campus_task_db
psql (17.10 (Homebrew))
도움말을 보려면 "help"를 입력하십시오.

campus_task_db=#
```

2.1 CREATE DATABASE / CREATE SCHEMA 실행

```
01-create-database.sql ✕
CampusTask > sql > 01-create-database.sql > ...
PostgreSQL에 연결
1 -- 실행 위치: 기본 postgres 데이터베이스
2 -- 이 파일은 최초 한 번만 실행합니다.
3
4 CREATE DATABASE campus_task_db
5 WITH ENCODING = 'UTF8'
6 ||| TEMPLATE = template0;
7
8

문제 출력 디버그 콘솔 터미널 포트 POSTGRESQL QUERY RESULTS
➤ + CampusTask git:(main) ✕ cd sql
➤ + sql git:(main) ✕ psql -v ON_ERROR_STOP=1 \
-d postgres \
-f 01-create-database.sql
CREATE DATABASE
```

[CREATE DATABASE]

```
02-create-schema.sql ✕
CampusTask > sql > 02-create-schema.sql > ...
PostgreSQL에 연결
1 -- 실행 위치: campus_task_db 데이터베이스
2
3 CREATE SCHEMA IF NOT EXISTS campus_task;
4
5 -- 현재 접속 세션에서 campus_task 스키마를 기본으로 사용합니다.
6 SET search_path TO campus_task, public;
7
8 SELECT current_database() AS database_name,
9        current_schema() AS schema_name;
10
11

문제 출력 디버그 콘솔 터미널 포트 POSTGRESQL QUERY RESULTS
➤ + sql git:(main) ✕ psql -v ON_ERROR_STOP=1 \
-d campus_task_db \
-f 02-create-schema.sql
CREATE SCHEMA
SET
-----
database_name | schema_name
-----
campus_task_db | campus_task
(1개 행)
```

[CREATE SCHEMA]

2.2 CREATE TABLE — 제약조건 포함 DDL 작성

```
03-create-tables.sql ✕
CampusTask > sql > 03-create-tables.sql > ...
PostgreSQL에 연결
1 -- 실행 위치: campus_task_db 데이터베이스
2
3 SET search_path TO campus_task, public;
4
5 -- 1. 학생
6 CREATE TABLE students (
7     id BIGINT GENERATED ALWAYS AS IDENTITY,
8     student_no VARCHAR(20) NOT NULL,
9     name VARCHAR(50) NOT NULL,
10    email VARCHAR(100) NOT NULL,
11    phone VARCHAR(20),
12    enrolled BOOLEAN NOT NULL DEFAULT TRUE,
13    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
14
15    CONSTRAINT pk_students PRIMARY KEY (id),
16    CONSTRAINT uk_students_student_no UNIQUE (student_no),
17    CONSTRAINT uk_students_email UNIQUE (email)
18 );
19
20 -- 2. 강의
21 CREATE TABLE courses (
22     id BIGINT GENERATED ALWAYS AS IDENTITY,
23     course_code VARCHAR(20) NOT NULL,
24     title VARCHAR(100) NOT NULL,
25
26     CONSTRAINT pk_courses PRIMARY KEY (id),
27     CONSTRAINT uk_courses_course_code UNIQUE (course_code)
28 );
29
30 -- 3. 수강신청: 학생과 강의의 N:M 관계를 연결하는 교차 테이블
31 CREATE TABLE enrollments (
32     student_id BIGINT NOT NULL,
33     course_id BIGINT NOT NULL,
34     enrolled_at DATE NOT NULL DEFAULT CURRENT_DATE,
35
36     CONSTRAINT pk_enrollments PRIMARY KEY (student_id, course_id),
37     FOREIGN KEY (student_id) REFERENCES students (id) ON DELETE CASCADE,
38     FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE
39 );
40
41 -- 4. 과제
42 CREATE TABLE assignments (
43     id BIGINT GENERATED ALWAYS AS IDENTITY,
44     course_id BIGINT NOT NULL,
45     title VARCHAR(200) NOT NULL,
46     due_at TIMESTAMPTZ NOT NULL,
47
48     CONSTRAINT pk_assignments PRIMARY KEY (id),
49     FOREIGN KEY (course_id) REFERENCES courses (id) ON DELETE CASCADE
50 );
51
52 -- 5. 과제 제출
53 CREATE TABLE submissions (
54     id BIGINT GENERATED ALWAYS AS IDENTITY,
55     assignment_id BIGINT NOT NULL,
56     student_id BIGINT NOT NULL,
57     submitted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
58     score NUMERIC(5,2),
59     feedback TEXT,
60
61     CONSTRAINT pk_submissions PRIMARY KEY (id),
62     CONSTRAINT uk_submissions_assignment_student UNIQUE (assignment_id, student_id),
63     CONSTRAINT chk_submissions_score CHECK (score IS NULL OR score BETWEEN 0 AND 100),
64     FOREIGN KEY (assignment_id) REFERENCES assignments (id) ON DELETE CASCADE,
65     FOREIGN KEY (student_id) REFERENCES students (id) ON DELETE CASCADE
66 );
67
68 -- 교차 테이블을 course_id부터 조회할 때 사용할 인덱스
69 CREATE INDEX idx_enrollments_course_student
70 ON enrollments (course_id, student_id);
71
72 -- 학생별 제출물을 빠르게 조회하기 위한 인덱스
73 CREATE INDEX idx_submissions_student
74 ON submissions (student_id);
75
76 -- 생성된 테이블 확인
77 SELECT table_name
78 FROM information_schema.tables
79 WHERE table_schema = 'campus_task'
80 ORDER BY table_name;
81
82 문제 출력 디버그 콘솔 터미널 포트 POSTGRESQL QUERY RESULTS
➤ + sql git:(main) ✕ psql -v ON_ERROR_STOP=1 \
-d campus_task_db \
-f 03-create-tables.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
table_name
-----
assignments
courses
enrollments
students
submissions
(5개 행)
```

```
03-create-tables.sql ✕
CampusTask > sql > 03-create-tables.sql > ...
PostgreSQL에 연결
54
55 -- 5. 과제 제출
56 CREATE TABLE submissions (
57     id BIGINT GENERATED ALWAYS AS IDENTITY,
58     assignment_id BIGINT NOT NULL,
59     student_id BIGINT NOT NULL,
60     submitted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
61     score NUMERIC(5,2),
62     feedback TEXT,
63
64     CONSTRAINT pk_submissions PRIMARY KEY (id),
65     CONSTRAINT uk_submissions_assignment_student UNIQUE (assignment_id, student_id),
66     CONSTRAINT chk_submissions_score CHECK (score IS NULL OR score BETWEEN 0 AND 100),
67     FOREIGN KEY (assignment_id) REFERENCES assignments (id) ON DELETE CASCADE,
68     FOREIGN KEY (student_id) REFERENCES students (id) ON DELETE CASCADE
69 );
70
71 -- 교차 테이블을 course_id부터 조회할 때 사용할 인덱스
72 CREATE INDEX idx_enrollments_course_student
73 ON enrollments (course_id, student_id);
74
75 -- 학생별 제출물을 빠르게 조회하기 위한 인덱스
76 CREATE INDEX idx_submissions_student
77 ON submissions (student_id);
78
79 -- 생성된 테이블 확인
80 SELECT table_name
81 FROM information_schema.tables
82 WHERE table_schema = 'campus_task'
83 ORDER BY table_name;
84
85 문제 출력 디버그 콘솔 터미널 포트 POSTGRESQL QUERY RESULTS
➤ + sql git:(main) ✕ psql -v ON_ERROR_STOP=1 \
-d campus_task_db \
-f 03-create-tables.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
table_name
-----
assignments
courses
enrollments
students
submissions
(5개 행)
```

2.3 INSERT INTO (테이블 데이터는 최소 10건 이상)

```
O4-insert-sample-data.sql x
CampusTask > sql > O4-insert-sample-data.sql > ...
PostgreSQL에 연결
-- 실행 위치: campus_task_db 데이터베이스
-- 03-create-tables.sql 실행 직후, 비어 있는 테이블에서 한 번 실행합니다.
SET search_path TO campus_task, public;

-- 학생 12건
INSERT INTO students (student_no, name, email, phone, enrolled, created_at) VALUES
  ('2026001', '김민준', 'minjun.kim@campus.test', '010-1001-1001', TRUE, '2026-03-02 09:00:00+09'),
  ('2026002', '이서연', 'seoyeon.lee@campus.test', NULL, TRUE, '2026-03-02 09:10:00+09'),
  ('2026003', '박지훈', 'jihun.park@campus.test', '010-1003-1003', TRUE, '2026-03-02 09:20:00+09'),
  ('2026004', '최유진', 'yujin.choi@campus.test', NULL, TRUE, '2026-03-02 09:30:00+09'),
  ('2026005', '정하늘', 'haneul.jeong@campus.test', '010-1005-1005', TRUE, '2026-03-02 09:40:00+09'),
  ('2026006', '강도윤', 'doyun.kang@campus.test', '010-1006-1006', TRUE, '2026-03-02 09:50:00+09'),
  ('2026007', '윤지아', 'jia.yoon@campus.test', NULL, TRUE, '2026-03-02 10:00:00+09'),
  ('2026008', '정현우', 'hyunwoo.jang@campus.test', '010-1008-1008', TRUE, '2026-03-02 10:10:00+09'),
  ('2026009', '임수빈', 'subin.lim@campus.test', '010-1009-1009', TRUE, '2026-03-02 10:20:00+09'),
  ('2026010', '한예준', 'yejun.han@campus.test', NULL, TRUE, '2026-03-02 10:30:00+09'),
  ('2026011', '오재원', 'chaewon.oh@campus.test', '010-1011-1011', FALSE, '2026-03-02 10:40:00+09'),
  ('2026012', '서준호', 'junho.seo@campus.test', NULL, TRUE, '2026-03-02 10:50:00+09');

-- 강좌 10건
INSERT INTO courses (course_code, title) VALUES
  ('DB101', '데이터베이스 기초'),
  ('JAVA101', 'Java 프로그래밍'),
  ('WEB101', '웹 개발 기초'),
  ('PYTHON101', 'Python 프로그래밍'),
  ('CLOUD101', '클라우드 기초'),
  ('AI101', '인공지능 입문'),
  ('DATA201', '데이터 분석'),
  ('DEVOPS101', 'DevOps 기초'),
  ('SEC101', '정보보안 기초'),
  ('PROJECT1', '팀 프로젝트');

-- 수강신청 24건
INSERT INTO enrollments (student_id, course_id, enrolled_at) VALUES
  (1, 1, '2026-03-03'), (1, 2, '2026-03-03'),
  (2, 1, '2026-03-03'), (2, 3, '2026-03-03'),
  (3, 1, '2026-03-03'), (3, 4, '2026-03-03'),
  (4, 2, '2026-03-03'), (4, 3, '2026-03-03'),
  (5, 2, '2026-03-03'), (5, 5, '2026-03-03'),
  (6, 3, '2026-03-03'), (6, 6, '2026-03-03'),
  (7, 4, '2026-03-03'), (7, 7, '2026-03-03'),
  (8, 5, '2026-03-03'), (8, 8, '2026-03-03'),
  (9, 6, '2026-03-03'), (9, 9, '2026-03-03'),
  (10, 7, '2026-03-03'), (10, 10, '2026-03-03'),
  (11, 8, '2026-03-03'), (11, 9, '2026-03-03'),
  (12, 9, '2026-03-03'), (12, 10, '2026-03-03');
```

```
O4-insert-sample-data.sql x
CampusTask > sql > O4-insert-sample-data.sql > ...
-- 과제 10건: 강좌별 1건
INSERT INTO assignments (course_id, title, due_at) VALUES
  (1, 'ERD 작성 과제', '2026-08-10 23:59:00+09'),
  (2, 'Java 클래스 설계', '2026-08-12 23:59:00+09'),
  (3, 'HTML/CSS 레이아웃 작업', '2026-08-13 23:59:00+09'),
  (4, 'Python 데이터 처리', '2026-08-14 23:59:00+09'),
  (5, '클라우드 구성도 작성', '2026-08-15 23:59:00+09'),
  (6, 'AI 활용 사례 조사', '2026-08-16 23:59:00+09'),
  (7, '데이터 분석 보고서', '2026-08-17 23:59:00+09'),
  (8, 'CI/CD 파이프라인 설계', '2026-08-18 23:59:00+09'),
  (9, '보안 취약점 점검', '2026-08-19 23:59:00+09'),
  (10, '프로젝트 요구사항 정리', '2026-08-20 23:59:00+09');

-- 과제 제출 12건: 정상 제출, 지각 제출, 미제출, 피드백 NULL을 함께 구성
INSERT INTO submissions (assignment_id, student_id, submitted_at, score, feedback)
VALUES
  (1, 1, '2026-08-10 20:15:00+09', 95.00, 'ERD 관계 표현이 명확합니다.'),
  (1, 2, '2026-08-11 00:30:00+09', 82.00, '마감 시간을 확인하세요.'),
  (1, 3, '2026-08-10 22:10:00+09', 90.00, NULL),
  (2, 1, '2026-08-12 21:00:00+09', 88.00, '클래스 분리가 적절합니다.'),
  (2, 4, '2026-08-13 01:20:00+09', 76.00, '지각 제출입니다.'),
  (2, 5, '2026-08-12 19:40:00+09', NULL, NULL),
  (3, 2, '2026-08-13 20:30:00+09', 92.00, '웹인 구성이 좋습니다.'),
  (3, 4, '2026-08-13 23:40:00+09', 85.00, NULL),
  (3, 6, '2026-08-14 00:10:00+09', NULL, NULL),
  (4, 3, '2026-08-14 10:00:00+09', 91.00, '코드가 간결합니다.'),
  (4, 7, '2026-08-15 02:00:00+09', 79.00, '데이터 처리를 보완하세요.'),
  (5, 5, '2026-08-15 20:00:00+09', NULL, NULL);

-- 테이블별 데이터 개수 확인
SELECT 'students' AS table_name, COUNT(*) AS row_count FROM students
UNION ALL
SELECT 'courses', COUNT(*) FROM courses
UNION ALL
SELECT 'enrollments', COUNT(*) FROM enrollments
UNION ALL
SELECT 'assignments', COUNT(*) FROM assignments
UNION ALL
SELECT 'submissions', COUNT(*) FROM submissions
ORDER BY table_name;
```

문제 출처: 다그 콘서트 | 테이블 | 포스트greSQL QUERY RESULTS

```
sql git:(main) # psql -v ON_ERROR_STOP=1 \
-d campus_task_db \
-f O4-insert-sample-data.sql
INSERT 0 12
INSERT 0 10
INSERT 0 24
INSERT 0 10
INSERT 0 12
table_name | row_count
--
assignments | 10
courses     | 10
enrollments  | 24
students     | 12
submissions  | 12
(5개 행)
```

2.4 SELECT + WHERE + ORDER BY 기초 조회

Filter connector: postgres localhost:54...

campus_task_db

Administer

System Info

실행 위치: campus_task_db 데이터베이스

SET search_path TO campus_task, public;

Q1. 재학 중인 학생을 학번 오름차순으로 조회하시오.
사용 문법: SELECT, WHERE, ORDER BY

```
SELECT
  student_no,
  name,
  email
FROM students
WHERE enrolled = TRUE
ORDER BY student_no ASC;
```

Q2. 학생 전화번호가 NULL이면 '미등록'으로 표시하시오.
사용 함수: COALESCE

```
SELECT
  student_no,
  name,
  COALESCE(phone, '미등록') AS phone_display
FROM students
ORDER BY student_no ASC;
```

Q3. 제출 시간과 마감 시간을 비교하여 정상 제출과 지각 제출을 표시하시오.
사용 함수: CASE, SUBSTR, TO_TIMESTAMP

```
SELECT
  assignment_id,
  student_id,
  submitted_at,
  score,
  CASE
    WHEN submitted_at <= due_at THEN '정상'
    ELSE '지각'
  END AS status
FROM submissions
ORDER BY assignment_id, student_id;
```

students 1 x

SELECT student_no, name, email FROM students

Enter a SQL expression to filter results (use Ctrl+Space)

AZ student_no	AZ name	AZ email
1	2026001	김민준
2	2026002	이서연
3	2026003	박지훈
4	2026004	최유진
5	2026005	정하늘
6	2026006	강도윤
7	2026007	윤지아
8	2026008	정현우
9	2026009	임수빈
10	2026010	한예준
11	2026011	오재원
12	2026012	서준호

Value x

2026001

11 row(s) fetched - 0.024s (0.001s fetch), on 2026-08-11 at 17:19:20

2.5 COALESCE / CASE WHEN / 날짜 함수 활용

The screenshot shows a PostgreSQL IDE interface. The script editor contains the following SQL code:

```
SET search_path TO campus_task, public;

-- Q1. 재학 중인 학생을 학번 오름차순으로 조회하시오.
-- 사용 문법: SELECT, WHERE, ORDER BY
SELECT
  student_no,
  name,
  email
FROM students
WHERE enrolled = TRUE
ORDER BY student_no ASC;

-- Q2. 학생 전화번호가 NULL이면 '미등록'으로 표시하시오.
-- 사용 함수: COALESCE
SELECT
  student_no,
  name,
  COALESCE(phone, '미등록') AS phone_display
FROM students
ORDER BY student_no ASC;

-- Q3. 제출 시간과 마감 시간을 비교하여 정상 제출과 지각 제출을 표시하시오.
-- 사용 문법: CASE WHEN, JOIN
SELECT
```

The query result is displayed in a table with 12 rows:

AZ student_no	AZ name	AZ phone_display
1	김민준	010-1001-1001
2	이서연	미등록
3	박지훈	010-1003-1003
4	최유진	미등록
5	정하늘	010-1005-1005
6	강도윤	010-1006-1006
7	윤지아	미등록
8	장현우	010-1008-1008
9	임수빈	010-1009-1009
10	한예준	미등록
11	오채원	010-1011-1011
12	서준호	미등록

[COALESCE]

The screenshot shows a PostgreSQL IDE interface. The script editor contains the following SQL code:

```
-- Q3. 제출 시간과 마감 시간을 비교하여 정상 제출과 지각 제출을 표시하시오.
-- 사용 문법: CASE WHEN, JOIN
SELECT
  s.student_no,
  s.name AS student_name,
  a.title AS assignment_title,
  a.due_at,
  sb.submitted_at,
  CASE
    WHEN sb.submitted_at <= a.due_at THEN '정상 제출'
    ELSE '지각 제출'
  END AS submission_status
FROM submissions AS sb
JOIN students AS s
ON s.id = sb.student_id
JOIN assignments AS a
ON a.id = sb.assignment_id
ORDER BY a.due_at ASC, s.student_no ASC;

-- Q4. 과제 마감 일에서 연도, 월, 일을 추출하시오.
-- 사용 함수: EXTRACT, TO_CHAR
SELECT
  title,
```

The query result is displayed in a table with 12 rows:

AZ student_no	AZ student_name	AZ assignment_title	due_at	submitted_at	AZ submission_status
1	김민준	ERD 작성 과제	2026-08-10 23:59:00.000 +0900	2026-08-10 20:15:00.000 +0900	정상 제출
2	이서연	ERD 작성 과제	2026-08-10 23:59:00.000 +0900	2026-08-11 00:30:00.000 +0900	지각 제출
3	박지훈	ERD 작성 과제	2026-08-10 23:59:00.000 +0900	2026-08-10 22:10:00.000 +0900	정상 제출
4	김민준	Java 클래스 설계	2026-08-12 23:59:00.000 +0900	2026-08-12 21:00:00.000 +0900	정상 제출
5	최유진	Java 클래스 설계	2026-08-12 23:59:00.000 +0900	2026-08-13 01:20:00.000 +0900	지각 제출
6	정하늘	Java 클래스 설계	2026-08-12 23:59:00.000 +0900	2026-08-12 19:40:00.000 +0900	정상 제출
7	이서연	HTML-CSS 페이지 작성	2026-08-13 23:59:00.000 +0900	2026-08-13 20:30:00.000 +0900	정상 제출
8	최유진	HTML-CSS 페이지 작성	2026-08-13 23:59:00.000 +0900	2026-08-13 23:40:00.000 +0900	정상 제출
9	강도윤	HTML-CSS 페이지 작성	2026-08-13 23:59:00.000 +0900	2026-08-14 00:10:00.000 +0900	지각 제출
10	박지훈	Python 데이터 처리	2026-08-14 23:59:00.000 +0900	2026-08-14 18:00:00.000 +0900	정상 제출
11	윤지아	Python 데이터 처리	2026-08-14 23:59:00.000 +0900	2026-08-15 02:00:00.000 +0900	지각 제출
12	정하늘	클라우드 구성도 작성	2026-08-15 23:59:00.000 +0900	2026-08-15 20:00:00.000 +0900	정상 제출

[CASE WHEN]

Q4. 과제 마감 일시에서 연도, 월, 일을 추출하십시오.
 사용 함수: EXTRACT, TO_CHAR

```
SELECT
  title,
  due_at,
  EXTRACT(YEAR FROM due_at) AS due_year,
  EXTRACT(MONTH FROM due_at) AS due_month,
  EXTRACT(DAY FROM due_at) AS due_day,
  TO_CHAR(due_at, 'YYYY-MM-DD HH24:MI') AS due_display
FROM assignments
ORDER BY due_at ASC;
```

Q5. 수강신청 교차 테이블을 이용하여 학생별 수강 강좌를 조회하십시오.
 사용 문법: students → enrollments → courses JOIN

```
SELECT
  s.student_no,
  s.name AS student_name,
  c.course_code,
  c.title AS course_title,
  e.enrolled_at
FROM students AS s
JOIN enrollments AS e
  ON e.student_id = s.id
```

AZ title	due_at	123 due_year	123 due_month	123 due_day	AZ due_display
1 ERD 작성 과제	2026-08-10 23:59:00.000 +0900	2,026	8	10	2026-08-10 23:59
2 Java 클래스 설계	2026-08-12 23:59:00.000 +0900	2,026	8	12	2026-08-12 23:59
3 HTML-CSS 페이지 작성	2026-08-13 23:59:00.000 +0900	2,026	8	13	2026-08-13 23:59
4 Python 데이터 처리	2026-08-14 23:59:00.000 +0900	2,026	8	14	2026-08-14 23:59
5 클라우드 구성도 작성	2026-08-15 23:59:00.000 +0900	2,026	8	15	2026-08-15 23:59
6 AI 활용 사례 조사	2026-08-16 23:59:00.000 +0900	2,026	8	16	2026-08-16 23:59
7 데이터 분석 보고서	2026-08-17 23:59:00.000 +0900	2,026	8	17	2026-08-17 23:59
8 CI/CD 파이프라인 설계	2026-08-18 23:59:00.000 +0900	2,026	8	18	2026-08-18 23:59
9 보안 취약점 점검	2026-08-19 23:59:00.000 +0900	2,026	8	19	2026-08-19 23:59
10 프로젝트 요구사항 정의	2026-08-20 23:59:00.000 +0900	2,026	8	20	2026-08-20 23:59

[날짜 함수 활용]

2.6 수강신청 교차 테이블 JOIN 조회

Q5. 수강신청 교차 테이블을 이용하여 학생별 수강 강좌를 조회하십시오.
 사용 문법: students → enrollments → courses JOIN

```
SELECT
  s.student_no,
  s.name AS student_name,
  c.course_code,
  c.title AS course_title,
  e.enrolled_at
FROM students AS s
JOIN enrollments AS e
  ON e.student_id = s.id
JOIN courses AS c
  ON c.id = e.course_id
ORDER BY s.student_no ASC, c.course_code ASC;
```

AZ student_no	AZ student_name	AZ course_code	AZ course_title	enrolled_at
1 2026001	김민준	DB101	데이터베이스 기초	2026-03-03
2 2026001	김민준	JAVA101	Java 프로그래밍	2026-03-03
3 2026002	이서연	DB101	데이터베이스 기초	2026-03-03
4 2026002	이서연	WEB101	웹 개발 기초	2026-03-03
5 2026003	박지훈	DB101	데이터베이스 기초	2026-03-03
6 2026003	박지훈	PYTHON101	Python 프로그래밍	2026-03-03
7 2026004	최유진	JAVA101	Java 프로그래밍	2026-03-03
8 2026004	최유진	WEB101	웹 개발 기초	2026-03-03
9 2026005	정하늘	CLOUD101	클라우드 기초	2026-03-03
10 2026005	정하늘	JAVA101	Java 프로그래밍	2026-03-03
11 2026006	강도윤	AI101	인공지능 입문	2026-03-03
12 2026006	강도윤	WEB101	웹 개발 기초	2026-03-03
13 2026007	윤지아	DATA201	데이터 분석	2026-03-03
14 2026007	윤지아	PYTHON101	Python 프로그래밍	2026-03-03
15 2026008	장현우	CLOUD101	클라우드 기초	2026-03-03
16 2026008	장현우	DEVOPS101	DevOps 기초	2026-03-03
17 2026009	임수빈	AI101	인공지능 입문	2026-03-03
18 2026009	임수빈	SEC101	정보보안 기초	2026-03-03
19 2026010	한예준	DATA201	데이터 분석	2026-03-03
20 2026010	한예준	PROJECT1	팀 프로젝트	2026-03-03
21 2026011	오재원	DEVOPS101	DevOps 기초	2026-03-03
22 2026011	오재원	SEC101	정보보안 기초	2026-03-03
23 2026012	서준호	PROJECT1	팀 프로젝트	2026-03-03
24 2026012	서준호	SEC101	정보보안 기초	2026-03-03

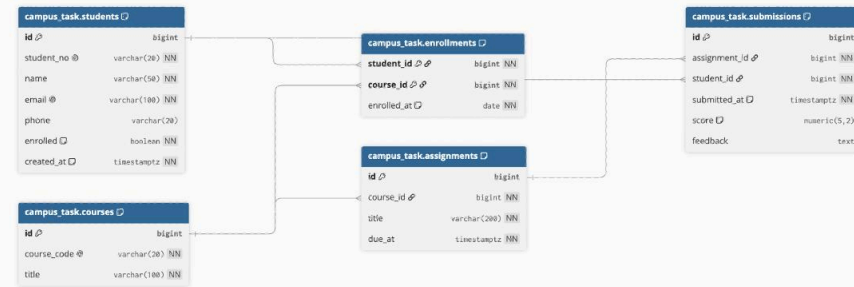
3. ERD

학사관리시스템 - 수강 및 과제관리 ERD

설계 규칙

- 테이블 컬럼명은 영문 소문자 snake_case, 테이블명은 복수형 사용
- 일반 테이블은 자동 증가 BIGINT id를 PK로 사용
- enrollments는 학생과 강좌의 N:M 관계를 연결하며, 두 FK를 복합 PK로 사용
- 모든 FK는 NOT NULL이며 부모 삭제 시 관련 데이터도 삭제
- 학번·이메일·강좌코드 및 학생별 과제 제출은 중복 불가
- 점수는 미채점이면 NULL, 채점 시 0~100점으로 제한
- 자주 조회하는 FK에는 인덱스 적용

표기	의미
TABLE_NAME	테이블명
id (PK)	기본키
normal_column	일반 속성
parent_id (FK)	외래키
column (UQ)	중복을 허용하지 않는 속성



범례(IE 표기법)

